Lesson 8 Teacher's Key: Under Pressure

Unity - Instructor Reference Only

VIDEO GAME CREATION

What This Lesson Teaches

A second static method (RegisterEscape()) added to GameManager alongside the existing RegisterKill() - reinforcing that a single manager class can safely host multiple, clearly-named, single-purpose methods without them interfering.

Level 1 - Full Worked Solution

- lives and hasLost are new static fields, parallel to enemiesRemaining and hasWon - same class, same pattern, different names.
- RegisterEscape() is called from the EXISTING escape branch in Enemy.cs's Update(), right before the pre-existing Destroy(gameObject) call - one new line in an already-correct location.
- OnGUI() gets a second if-block for hasLost, alongside the existing hasWon block - two independent checks, two independent messages.

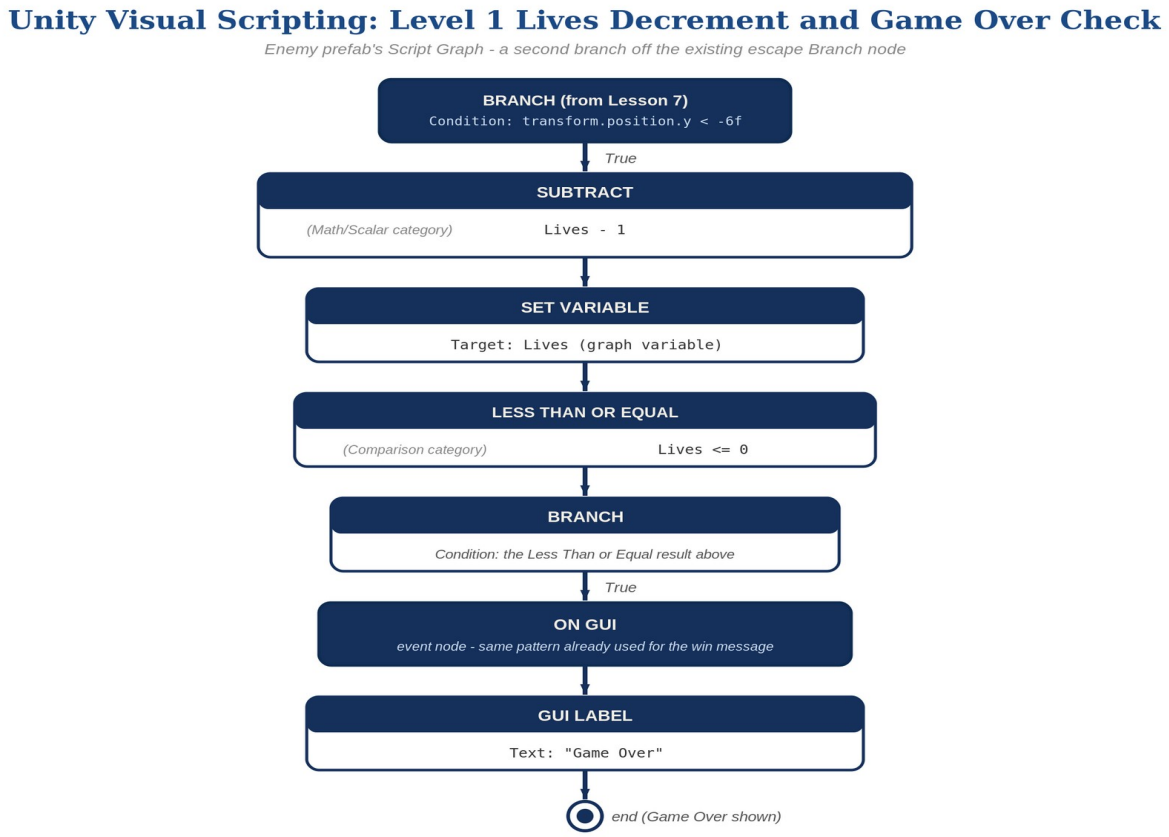
Highlighted lines are new this week; everything else is code you already have.

```
// GameManager.cs - add fields and a new method:
public static int lives = 3;
public static bool hasLost = false;

public static void RegisterEscape() {
    lives--;
    if (lives <= 0) hasLost = true;
}

// GameManager.cs's OnGUI() - add alongside the hasWon check:
if (hasLost) {
    GUI.Label(new Rect(Screen.width/2 - 50, 90, 200, 30), "Game Over");
}

// Enemy.cs's Update() - escape branch (revised):
if (transform.position.y < -6f) {
    GameManager.RegisterEscape();
    Destroy(gameObject);
}
```



The Enemy prefab's Script Graph - the node-for-node equivalent of the Level 1 lives-tracking logic.

Why This Structure

- This is the direct payoff of Lesson 7's fix: RegisterKill() lives exclusively in Bullet.cs, and now RegisterEscape() lives exclusively in Enemy.cs's own Update() - two methods, two call sites, each unambiguous about what it means.
- Keeping both methods on the same GameManager class (rather than splitting into multiple manager scripts) is fine here because the class itself isn't doing the deciding - each calling script decides which method applies to its own situation.

Level 2 / Level 3 Talking Points

- Level 3 sound, worked (Unity): GameManager.cs gets public AudioClip lifeLostSound and gameOverSound, assigned in the Inspector from the provided sfx_lifelost.wav / sfx_gameover.wav. Because RegisterEscape() is static and the clips are instance fields, GameManager also needs a static instance reference set in Awake() (see the Level 3 code below). In RegisterEscape(): lives--, then if (lives <= 0 && !hasLost) { hasLost = true; AudioSource.PlayClipAtPoint(gameOverSound, Camera.main.transform.position); } else AudioSource.PlayClipAtPoint(lifeLostSound, Camera.main.transform.position). Visual Scripting path: the same two Play Clip At Point-style calls on the Branch's True/False outputs.

- Ask students: what's the actual difference between RegisterKill() and RegisterEscape(), code-wise? (One decrements enemiesRemaining and sets hasWon; the other decrements lives and sets hasLost - genuinely parallel, unrelated pieces of state.)
- Level 3 structure talking point: this GameManager class is starting to look like a real "game state" hub - ask students to predict what else might eventually live here (score, wave number, difficulty) as a preview of Lesson 9 and beyond.

Level 2 - complete code:

```
// Level 2 is verification only - the Level 1 code is already correct, so
no lines change.
// Checks: shooting an enemy never changes lives (only the kill path
changes enemies_remaining);
// an escape never changes enemies_remaining (only the escape path changes
lives);
// "Game Over" appears once, when lives first reaches 0.
```

Level 3 - complete code (all three goals combined):

Highlighted lines are the Level 3 additions on top of the Level 1 code.

```
// GameManager.cs - Level 3 (assign lifeLostSound and gameOverSound in the
Inspector):
public static int lives = 3;
public static bool hasLost = false;
public static GameManager instance;
public AudioClip lifeLostSound;
public AudioClip gameOverSound;

void Awake() {
    instance = this;
}

public static void RegisterEscape() {
    lives--;
    if (lives <= 0 && !hasLost) {
        hasLost = true;
        AudioSource.PlayClipAtPoint(instance.gameOverSound,
Camera.main.transform.position);
    } else if (lives > 0) {
        AudioSource.PlayClipAtPoint(instance.lifeLostSound,
Camera.main.transform.position);
    }
}

// GameManager.cs's OnGUI():
GUI.Label(new Rect(20, 20, 200, 30), "Lives: " + lives);
if (hasLost) {
    GUI.Label(new Rect(Screen.width/2 - 50, 90, 200, 30), "Game Over");
}
```

Common Student Mistakes

- Calling RegisterEscape() from Bullet.cs, or RegisterKill() from Enemy.cs - either one causes cross-contamination between lives and enemiesRemaining.
- Declaring hasLost as a non-static field by mistake (compiles fine but never updates the way hasWon does), copied incorrectly from the static hasWon declaration.

- OnGUI() messages overlapping on screen if hasWon and hasLost happen to become true around the same time - remind students this is a display/layout detail, not a logic bug.

Alternate Path Notes: Visual Scripting

The Enemy prefab's graph gets a second branch off its existing escape Branch node: a "Subtract" node (Math/Scalar category) computing Lives minus 1, feeding a "Set Variable" node that writes it back to the Lives graph variable - the node-graph equivalent of lives--.

Lives feeds a "Less Than or Equal" node (Comparison category) against 0, feeding a second Branch node. Its True output feeds an "On GUI" event node with a GUI Label showing "Game Over" - the exact same pattern Lesson 7's win message already used, just gated on a different variable.

Lives is its own graph variable, entirely separate from the enemies-remaining/win-tracking variable - the same separation RegisterKill()/RegisterEscape() enforce in the C# path, just expressed as two variables instead of two methods.

- This mirrors the C# path's structure exactly: one new decrement-and-check sequence, attached to the same escape Branch that already existed, touching only the Lives variable.
- Point out to students moving between C# and Visual Scripting that "static method call" and "node sequence wired to an event" are two different surfaces for the identical idea: a single, named place where a specific consequence happens.